Ders 10 Genişletilmiş Notu: Kod Kusurları: Divergent Change, Shotgun Surgery, Primitive Obsession, Feature Envy ve Video Kiralama Refactoring Örneği

Ders 10 Genişletilmiş Notu: Kod Kusurları: Divergent Change, Shotgun Surgery, Primitive Obsession, Feature Envy ve Video Kiralama Refactoring Örneği

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Divergent Change ve Shotgun Surgery
- 2. Primitive Obsession (İlkel Tiplerin Aşırı Kullanımı)
- 3. Replace Data Value with Object
- 4. Replace Type Code with Class
- 5. Replace Type Code with Subclasses
- 6. Replace Type Code with State/Strategy
- 7. Replace Array with Object
- 8. Feature Envy (Özellik Kıskançlığı)
- 9. Video Kiralama Örneğinde Ardışıl Refactoring
- 10. Kod Kusurlarından Anti-Pattern'lere
- Örnek Sorular
- Dersin Toparlaması

1. Divergent Change ve Shotgun Surgery

Ders, refactoring konusunun son birkaç kod kusurunu ve onları düzelten refactoring eylemlerini görmekle başladı. Ardından bütüncül bir refactoring örneği yapılacağı söylendi. Geçen derste Duplicated Code, Long Method, Long Parameter List ve Large Class işlenmişti.

Hoca bu iki kusurun adlarının biraz esprili olduğunu söyledi ve ikisini "değişikliklerin farklı noktalarda yapılmasının gerekmesi" başlığı altında iki kardeş kod kusuru olarak anlattı (ekrandan, 00:10:00).

Gereksinimlerde bir düzenleme olduğunda ya da bir hatayı düzeltmek gerektiğinde kodda değişiklik yapılır. Kod iyi tasarlanmışsa bu değişiklikler mümkün olduğunca az yerde yapılır. Bir gereksinim değişikliğini karşılamak için gereken kod değişiklikleri aynı sınıfın farklı metotlarında yapılıyorsa buna **Divergent Change**, birden fazla sınıfta yapılıyorsa buna **Shotgun Surgery** denir. İkisi de "bu işte bir terslik var" demektir.

1.1 Hocanın benzetmesi

Tasarımı gösteren sınıf şemasını açıp değişiklik yapılan yerleri işaretlediğinizi düşünün. Bütün değişiklikler aynı sınıfta kaldıysa durum yine iyi değildir ama Shotgun Surgery'den iyidir. Değişiklikler farklı farklı sınıflara yayılmışsa işaretlenen noktalar bir çifte (av tüfeği, "shotgun") atışına benzer: çiftenin fişeğindeki küçük saçmalar patladığında çok geniş bir alana yayılır.

Bu, çifteyle ameliyat yapmak gibidir; oysa ameliyat bıçakla yapılır. Divergent Change'de en azından tek bir bıçak vardır, ama aynı organın birçok yerine dokunulur; bu da iyi değildir.

> **Hocanın vurgusu:** Değişiklik mümkün olan en az yerde yapılmalıdır.

1.2 Örnekler

Divergent Change örneği: Uygulama bir veritabanı sistemini destekliyordu; lisans maliyetleri yüzünden başka bir veritabanına geçmek istendi ve bunun için bir sınıfın 3, 4, 5 ayrı metodunda değişiklik yapmak gerekti (slaytta: "Yeni bir veritabanı ile çalışmak gerekirse şu sınıfın şu 3 metodu değişir"). Doğru tasarımda, örneğin Strategy kalıbı kullanılmış olsaydı, yeni veritabanı için yeni bir sınıf eklemek ve üst sınıfın bir iki yerini değiştirmek yeterdi. Bütün veritabanı işlemleri tek bir sınıfa tıkıştırıldıysa, o sınıfın uyumu düşükse, sınıf "large class" ise ya da kopyala-yapıştır kod varsa, birçok yerde değişiklik yapmak zorunda kalınır.

Shotgun Surgery örneği: Kredi veren bir finansal program yapılmış olsun. Yeni bir kredi türü (paketi) işin içine girdiğinde birkaç ayrı sınıftaki birkaç ayrı metodu değiştirmek gerekiyorsa (slaytta: "şu 2 sınıftaki şu 5 metot değişmeli") bu Shotgun Surgery'dir. İstenen, bir iki metotla değişiklik yapılması ya da yeni bir sınıfın mümkün olan en az çabayla eklenmesidir. Tasarım kalıplarının sağladığı da budur.

1.3 Çözüm

Böyle bir noktaya gelindiyse bu iki kod kusurundan biri gerçekleşmiştir ve tasarımı toparlamak gerekir:

- **Extract Class:** Değişiklik yapmak zorunda kalınan metotlar ve bu metotların ağırlıklı olarak üzerinde çalıştığı üye alanlar mevcut yerlerinden çıkarılıp yepyeni bir sınıf oluşturulur.
- **Move Method / Move Field:** Yeni sınıf çıkarmak yerine bu üyeler, değişikliğin çapını azaltacak biçimde daha uygun, mevcut başka bir sınıfa taşınır.

Hoca bu noktada Move Method'u da açıkladı: Bir metot ait olduğu sınıfın içinden az (örneğin 3 noktadan), belirli bir başka sınıftan çok (örneğin 10 ayrı metottan) çağrılıyorsa, bu metoda aslında o diğer sınıfın daha çok ihtiyacı vardır. Bu durum bir yazılım görselleştirme aracıyla fark edilebilir. Metot onu daha çok çağıran sınıfa taşınır. Gerekli üye alanlar da taşınabilir ya da taşınmadan getter metotları üzerinden erişilebilir; asıl taşınması gereken metottur. Hoca bu durumun ileride göreceğimiz Feature Envy kusuruyla ilgili olduğunu söyledi. Extract Class yapılırken de ister istemez bazı metotlar yeni sınıfa taşınır; yani Extract Class kendi içinde başka eylemleri de barındırabilir.

2. Primitive Obsession (İlkel Tiplerin Aşırı Kullanımı)

Primitive tipler ilkel tiplerdir (`int`, `double`, `String` gibi). "Obsession" takıntı demektir; hoca bunu "gerektiğinden fazla kullanmak" anlamında "aşırı kullanım" diye çevirmeyi tercih etti (ekrandan, 00:16:30). Bu kusura özellikle yapısal programlamayla uzun ve yoğun uğraşmış yazılımcılar düşer.

Bu aşırı kullanımın farklı biçimleri vardır ve her birini kelimelerle tarif etmek zordur. Farklı biçimlerin farklı çözümleri olduğu için anlatım çözümler, yani refactoring eylemleri üzerinden yapıldı: "ne yapmışız, ne yapmalıydık". Çözümlerin hepsinin adı "Replace" ile başlar:

- Replace Data Value with Object
- Replace Type Code with Class
- Replace Type Code with Subclasses
- Replace Type Code with State/Strategy
- Replace Array with Object
- Extract Class ve Introduce Parameter Object da daha nesneye yönelimli bir program hazırlamak için kullanılabilir.

Type code (tip kodu) tek bir şekilde yapılan bir hatadır, ama mevcut tasarım hangisini gerektiriyorsa üç farklı biçimde düzeltilebilir: yeni bir sınıfla, alt sınıflarla ya da State/Strategy

kalıbıyla. Primitive Obsession'ın çok özel bir türü de nesne kullanmak yerine dizi ya da birden fazla dizi kullanmaktır; buna Replace Array with Object uygulanır.

Hoca Introduce Parameter Object'i hatırlattı: Bir metodun imzasında çok fazla parametre varsa ve bu parametre öbekleri birden fazla yerde birlikte geçiyorsa, bunlar birbiriyle ilişkili bir varlığı oluşturur ve onlardan yeni bir sınıf oluşturulur. Bu da ilkel tiplerin aşırı kullanımını azaltmaya yardımcı olur.

3. Replace Data Value with Object

Bir üye alan, hele ki bu alanla ilgili çeşitli davranışlar söz konusuysa, bir sınıfla simgelenmelidir (ekrandan, 00:20:00). Yani bir sınıfla modellenmesi gereken bir bilgi basitçe bir üye alan olarak modellenmişse bu eylem uygulanır.

Örnek: `Order` (sipariş) sınıfında siparişi veren müşterinin adı ya da tanımlayıcısı `customer: String` olarak tutulmuştur. Bu durum veritabanı odaklı düşünmekten kaynaklanıyor olabilir: belki bir müşteri tablosu vardır ve bu string ona yabancı anahtardır. Ama nesneye dayalı tasarımda bu doğru değildir. Müşteri başlı başına bir varlıktır; gereksinimlere bakıldığında müşteri hep ayrı bir varlık olarak görülür. İyi bir tasarımda problem aynı düzeyde parçalara ayrılır ve madem müşteri başlı başına bir parçadır, ayrı bir sınıf olmalıdır.

```
ÖNCE:                          SONRA:
Order                          Order ◆-----------------> 1 Customer
  customer: String                                         name: String
```

Daha kötüsü, `Order` içinde müşteriyle ilgili başka üye alanlar da oluşturulmuş olabilir. Müşteriler ayrı tutulmazsa, belli bir müşterinin siparişlerini bulmak için bütün sipariş listesinde tek tek sıralı arama (sequential search) yapmak gerekir; bu daha düşük başarımlı bir çalışmadır. Programın tümü incelendiğinde belli bir müşterinin tüm siparişlerinin aranması gibi işlemlerin yapıldığı görülecektir. Bu işlemler çoğaldıkça, müşteri sınıfı ayrılmadığı sürece sipariş sınıfı aşırı büyüyecek, uzun metotlar oluşacaktır. İyisi mi şimdiden bu sınıfları ayırmaktır. `Order` içindeki müşteriyle ilgili diğer üyeler de `Customer` sınıfına taşınmalıdır.

Hocanın slayta itirazı: Hoca slayttaki ilişkinin yönünü ve elmasın yerini sorguladı. Bir müşteri birden fazla sipariş verebildiği için bu bir parça-bütün ilişkisidir ve elmas bütün tarafında, yani `Customer` tarafında olmalıdır; ok da ters çizilmiştir. Sonra şunu da düşündü: belki daha önce müşteri sınıfı yoktu ve işler hep siparişler üzerinden yürüyordu; ya da aynı siparişi birden fazla müşteri veriyor olabilir. Yine de ilişkinin ters modellenmesi gerektiğini düşündüğünü söyleyip konuyu kendine not aldı. Asıl ders şudur: müşteri başlı başına bir varlık olarak bir sınıf şeklinde modellenmelidir.

4. Replace Type Code with Class

Bu eylem yapısal programlamada sabit değişkenler kullanma alışkanlığıyla ilgilidir (ekrandan, 00:24:40).

Hocaya göre burada en güzel çözüm `enum` kullanmaktır. Bir bilgi sabit ve değişmez birkaç seçenek şeklinde modellenebiliyorsa `enum` kullanılır. Örnekler:

- Kan grubu (Rh faktörü düşünülmeden, yalnız ana grup): 0, A, B, AB. Bunların dışında, örneğin "C" diye bir kan grubu olamaz.
- Satılan kıyafet ya da pizza boyları: small, medium, large (kıyafette x-large gibi seçenekler de olabilir), ama sınırlı ve bellidir.

Kullanılan nesneye yönelik dil `enum` desteklemiyorsa (düşük bir olasılık) ya da `enum` tercih edilmemişse, kötü tasarım şöyle olur: `Person` sınıfına `O`, `A`, `B`, `AB` adında dört `public static final int` üye alan (sabit) tanımlanmış, kodda büyük olasılıkla `O`'ya 0, `A`'ya 1, `B`'ye 2, `AB`'ye 3 değeri verilmiştir. Kişinin kan grubu `bloodGroup: int` üye alanında tutulur ve bu alana sabitlerden biri atanır.

```
ÖNCE:                      SONRA:
Person                     Person
  O: int     (static)        |
  A: int     (static)        | 1
```

```
  B: int     (static)        v
  AB: int    (static)      BloodGroup
  bloodGroup: int            O: BloodGroup   (static)
                             A: BloodGroup   (static)
                             B: BloodGroup   (static)
                             AB: BloodGroup  (static)
```

(ekrandan, 00:24:40; UML'de altı çizili üyeler static'tir.)

Bu doğru değildir; kan grubu bilgisi başlı başına bir varlıktır. Kan grubuyla ilgili kısım Extract Class ile `BloodGroup` diye yeni bir sınıfa çıkarılmalı, sabitler orada tanımlanmalı, `Person` da `BloodGroup` türünden bir üye alana sahip olmalıdır. Hoca, sınıfa verilecek adın ipucunun üye alanın adında olduğunu söyledi.

Bu kusurlu kullanım başka hatalara da yol açar: örneğin `bloodGroup` alanına 5 değeri atanabilir. Bu yüzden ya hemen `enum` kullanmaya geçilmeli ya da bu kısım ayrı sınıfa çıkarılmalı, gerekiyorsa kişiye kan grubu atanırken setter metotlarına ek kontroller konmalıdır. Slaytta da "Ya da kısaca enum kullanabilirsiniz" yazıyordu; hoca en iyisinin doğrudan `enum` kullanmak olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Java'da `enum BloodGroup { O, A, B, AB }` biçiminde tanımlanan bir tür, yalnız bu dört değeri alabilir; derleyici 5 gibi geçersiz bir değerin atanmasına izin vermez. Bu yüzden `enum`, Replace Type Code with Class'ın dile gömülü hali gibi düşünülebilir.

5. Replace Type Code with Subclasses

Bu durumda kalıtım hiç bilinmiyordur ya da "kalıtımı mümkün olduğunca az kullanalım" gibi bir söz kulağa çalınmış ve kalıtım gereken yerde kullanılmamıştır (ekrandan, 00:30:00).

`Employee` sınıfında farklı türden çalışanlar (mühendis, satış elemanı, sonradan eklenen yönetici) için `public static final int` sabitler ve çalışanın türünü tutan bir `_type` alanı tanımlanmıştır. Bu, kan grubu örneğinden çok farklı görünmez; ama modellenen olgu farklı olduğu için yapılması gereken refactoring eylemi tip kodunu alt sınıflarla değiştirmektir.

Önceki halde `Employee` sınıfının hangi metoduna bakılırsa bakılsın (iş yap, terfi et, maaş hesapla, prim hesapla...), içinde `switch(_type)` ya da `if (type == ...) ... else if (type == ...)` biçiminde karışık kodlar görülür.

> **Hocanın vurgusu:** Dersin en başında da söylendiği gibi, kendinizi `switch case` ya da `if else if` kullanırken görüyorsanız büyük ihtimalle kalıtımı ve onun getirdiği çok biçimlilik özelliğini kullanmanız gereken bir yerde kullanmamışsınız demektir.

```
/*** ÖNCE: ***/
class Employee {
  private int _type;
  static final int ENGINEER = 0;
  static final int SALESMAN = 1;
  static final int MANAGER = 2;
  Employee (int type) {
    _type = type; }
  public void work( ) {
    switch(_type) {
      case ENGINEER:
        //do something
        break;
      case SALESMAN:
        //do something
        break;
      case MANAGER:
        //do something
        break;
    }
```

```
  }
}

/*** SONRA: ***/
abstract class Employee {
  public abstract void work(){};
}
public class Engineer extends
    Employe {
  public void work( ) {
    //do something as an engineer
  }
}
public class Salesman...
```

(ekrandan, 00:30:00)

> **Ders sonrası not:** Slayttaki `public abstract void work(){};` satırı Java'da derlenmez; soyut bir metodun gövdesi olamaz. Doğrusu `public abstract void work();` olmalıdır. `extends Employe` satırında da `Employee` adı eksik yazılmıştır.

Hoca, her metodun üç durumlu bir `switch` içerdiği önceki halin yaklaşık üç kat uzun metotlar ürettiğini söyledi. Doğru tasarımda farklı çalışan türleri `Employee`'nin alt sınıfları olarak oluşturulur. Tüm türlerin ortak yaptığı işler varsa o metotlar `Employee`'de kalır.

- Mühendis ya da satış elemanı olmayan genel bir çalışan türü varsa bu genel tür `Employee` sınıfında modellenebilir; o zaman `Employee` normal (somut) bir sınıf olur. Slayttaki not da bunu söyler: önceki halde `case default` varsa sonraki halde `Employee` soyut yapılmaz.
- "Her çalışan illa belirli bir türden olmalı" deniyorsa, genel bir çalışan türü olsa bile o da `GenericEmployee` gibi ayrı bir alt sınıf olur ve `Employee` soyut sınıf olarak tanımlanır.

Slayttaki ikinci not: bu örnek **Replace Conditional with Polymorphism** düzenlemesine de benzer. Hoca burada yeni bir iş mantığıyla yeni sınıflar oluşturulduğu için ikisinin ilişkilendirilebileceğini söyledi.

6. Replace Type Code with State/Strategy

Önceki örnekte kalıtım bir nedenle doğrudan kullanılamıyorsa State ya da Strategy tasarım kalıbı kullanılabilir (ekrandan, 00:32:20).

```
ÖNCE:                  SONRA:
Employee               Employee ------------> 1 Employee Type
  ENGINEER: int                                      ^
  SALESMAN: int                                      |
  type: int                                 Engineer   Salesman
```

Hoca önce kalıtımın neden kullanılamayacağına dair bir senaryo bulmakta zorlandı. Kullanılan dil kalıtımı desteklemiyorsa bu çözüm de olmaz, çünkü State ve Strategy kalıpları da kalıtım gerektirir. Sonra asıl avantajı şöyle açıkladı:

Bir çalışan mühendis olarak işe alınmış olsun. Uzmanlık alanıyla ilgili ürünlerin satışı da söz konusu olunca bu mühendis ürün geliştirme departmanından satış departmanına geçirilebilir. Alt sınıf çözümünde bunu yapmak için yeni bir `Salesman` nesnesi oluşturmak, bütün bilgileri ilgili `Engineer` nesnesinden alıp kopyalamak ve eski `Engineer` nesnesini silmek gerekir. Kimlik numarası gibi bilgiler aynı kalacağı için bilgi kaybı olmasa da nesne kimliği (object identity) değişmiş olur. Nesne kimliği çok önemli değilse bu yapılabilir ama iş daha masraflıdır. State/Strategy tasarımında ise aynı `Employee` nesnesi, "mühendis durumundan satış elemanı moduna geçti" diye yalnız bağlı olduğu `Employee Type` nesnesi değiştirilerek kullanılmaya devam eder.

> **Hocanın vurgusu:** Bu esneklik gerekiyorsa Strategy (ya da State) kalıbı kullanılır; gerekmiyorsa düz çok biçimlilik (alt sınıflar) yeter.

7. Replace Array with Object

Hoca bu konuya lisans düzeyinde programlama öğretirken, öğrenciler henüz `struct` yapısını görmeden değindiğini söyledi (00:34:29).

Örnek: bir ligdeki takımların adlarını, puanlarını ve averajlarını tutmak istiyoruz. Hoca bunu ekranda bir tabloya yazarak gösterdi (takım adları örneğin Liverpool, Manchester United).

Kusurlu çözümler:

- **İki boyutlu dizi:** Her satır bir takım; ilk sütun takım adı, ikinci sütun puan, üçüncü sütun averaj (ya da tablonun transpozu). Veri türleri farklı olduğu için ve `String` daha kapsayıcı olduğu için puanlar da `String` olarak tutulur. Sonra her puan ya da averaj işleminde kullanılan dilin string'ten sayı oluşturan komutunu tekrar tekrar çağırmak gerekir.
- **Üç ayrı tek boyutlu dizi:** Takım adları (`String`), puanlar (`int`), averajlar (`int`). Bu durumda üç farklı dizide aynı indisli verilerin aynı takıma ait olduğunu bilmek ve kodu buna göre yazmak gerekir.

Hoca ikisinin de kötü olduğunu söyledi. Yapısal programlama dillerinde bile, örneğin Pascal'da `record`, C'de `struct` vardır. Böyle diziler kullanmak ancak ilgili yapılar bilinmiyorsa yapılacak şeydir.

Çözüm: `Team` ya da `Score` gibi (odak noktası neyse) bir sınıf oluşturulur; üye alanları takımın adı, galibiyet/puan ve averajdır. Dizi bir nesneye çevrilir ve o nesne türünden bir dizi kullanılır.

8. Feature Envy (Özellik Kıskançlığı)

Dersin son kod kusuru Feature Envy'dir (ekrandan, 00:40:00). Hoca bunun Shotgun Surgery ile bir paralelliği olduğunu söyledi.

Bir metot ait olduğu sınıfın üyelerinden çok başka sınıfların üyelerine erişiyorsa bu duruma "özellik kıskançlığı" adı verilir.

- "Çok" sıfatı ile o koddaki ortalama (ya da %20 gibi bir eşikten fazla) farklı sınıf üyesi erişim sayısının üzerinde bir sayı ifade edilir. Hoca bu eşiğin ciddi bir araştırma gerektirdiğini, ama istenirse bir sistemdeki sınıflar üzerinde hesaplanarak bulunabileceğini söyledi.
- Sorumlulukların, yani üye alanların ve metotların sınıflara doğru dağıtılmadığının göstergesi olabilir.
- Elbette bağlaşım ilkesi nedeniyle başka sınıflardan nesnelere erişmek gerekecektir. Bir sınıf işbirliği yaptığı sınıfların nesnelerinden metot (ve get metodu) çağıracaktır; bu yüzden hiçbir sınıf yalnız kendi metotlarını kullanmaz. Öyle olsaydı bu kez uyum ilkesi olumsuz etkilenirdi. Kusur, kendi üye alanlarına erişimden çok başka sınıfların üye alanlarına erişim olduğunda vardır.
- Strategy ve Visitor kalıpları bir özellik kıskançlığı durumu olarak yorumlanmamalıdır. Strategy'de yapılan bir işin farklı yapılma biçimleri olduğu için iş bilerek başkasına yaptırılır. Visitor'da da işlem ziyaret edilen nesnelerin üyeleri üzerinde yapılır. Hoca Visitor'ı daha önce eleştirdiklerini de hatırlattı, ama bu kalıplar bir kod kusuru değildir.

Çözüm için olası refactoring eylemleri:

- **Move Method:** Metot, kullandığı verinin bulunduğu sınıfa taşınır.
- **Extract Method:** Kıskançlık metodun yalnız bir kısmındaysa o kısım çıkarılıp kıskanılan üyelerin bulunduğu sınıfa taşınır.

Slaytın altında bir sonraki konunun başlığı vardı: "Bir tasarımın ardışıl düzenlemeler ile değiştirilmesi". Şimdiye kadar incelenen düzenlemelerden bazılarının birlikte kullanılarak yapısal programlama izleri taşıyan bir tasarımın nasıl değiştirildiğinin örneği için Fowler'ın Refactoring kitabının 1. bölümü incelenebilir.

9. Video Kiralama Örneğinde Ardışıl Refactoring

Aradan sonra hoca, şimdiye kadar görülen refactoring eylemlerini kullanarak kötü tasarlanmış bir uygulamanın adım adım nasıl kusursuz hale getirileceğini gösterdi. Örnek, refactoring anlatılmaya başlanmadan önce JUnit incelenirken görülen video kiralama uygulamasıdır. Hoca

ders notlarıyla ve Fowler'ın kitabıyla paralel gitti, arada notlardaki sıraya bakarak kodu Eclipse'te canlı olarak düzenledi.

9.1 Başlangıç tasarımı

- `Customer`: kiralamaları yapan müşteri. Bir müşterinin n adet kira kaydı vardır.
- `Rental`: kira kaydı. Her kira kaydı bir filmle ilişkilidir. Filmin kaç gün kiralandığı bilgisi bu sınıftadır.
- `Movie`: kiralanan film. Bir fiyat kodu (price code) vardır. Üç tür film vardır: normal filmler (`REGULAR`), yeni çıkan filmler (`NEW_RELEASE`) ve çocuk filmleri (`CHILDRENS`). Belli bir ücretle film belli bir süre kiralanır; o süre geçince her gün için ek ücret ödenir. Süreler ve ücretler film türüne göre değişir.
- `Customer.statement()`: müşterinin kira makbuzunu (rental record) üretip döndürür.

Hoca iş mantığı kodunu ve sınama kodunu farklı dizinlerde, ama aynı paket adlarıyla tuttuğunu gösterdi: `refactoring` dizini altında `fowler_00`, `fowler_01`... paketleri, `testing` dizini altında da aynı adlı paketlerde test sınıfları vardır. Eclipse'in bu yapılandırma olanağıyla test kodu ile uygulama mantığı ayrı dizinlerde durur (ekrandan, 01:10:00). Hoca geçici bir proje oluşturup `fowler_00` paketini ve test sınıflarını (`TestAll`, `TestCustomer`, `TestMovie`) oraya kopyaladı ve projeye JUnit 5 kütüphanesini ekledi (ekrandan, 01:20:00).

9.2 Nereden başlamalı?

Sınıflar incelendiğinde `Movie` ve `Rental` nispeten küçüktür; `Customer` ise büyüktür. `Customer` 57 satırdır, diğerleri 16 ve 22 satırdır; yani `Customer` diğer ikisinin toplamı kadardır. Kod satırı yerine başka ölçülere bakılsa uçurum daha da büyük görünür. Bu yüzden refactoring'e `Customer` ile başlamak en mantıklı seçimdir.

`Customer` içinde yapıcı ve get metotları küçüktür, ama `statement()` metodu 17. satırdan 56. satıra kadar yaklaşık 40 satırlık bir metottur. Hoca hangi kusurdan başlanacağının kişiden kişiye değişebileceğini, doğaçlama yapılsa farklı yerlerden başlanabileceğini, ama kitapla paralel gideceğini söyledi. `statement()` metodundaki en büyük sorun içindeki `switch case` bloğudur (ekrandan, 01:06:30):

```
public String statement( ) {
    double totalAmount = 0;
    int frequentRenterPoints = 0;
    Enumeration<Rental> rentals = _rentals.elements();
    String result = "Rental Record for " + getName() + "\n";
    while (rentals.hasMoreElements()) {
        double thisAmount = 0;
        Rental each = rentals.nextElement();
        //determine amounts for each line
        switch (each.getMovie().getPriceCode()) {
        case Movie.REGULAR:
            thisAmount += 2.0;
            if (each.getDaysRented() > 2)
                thisAmount += (each.getDaysRented() - 2) * 1.5;
            break;
        case Movie.NEW_RELEASE:
            thisAmount += each.getDaysRented() * 3.0;
            break;
        case Movie.CHILDRENS:
            ...
        }
        // add frequent renter points
        frequentRenterPoints ++;
        // add bonus for a two day new release rental
        if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE)
                && each.getDaysRented() > 1) frequentRenterPoints ++;
        //show figures for this rental
```

```
        result += "\t" + each.getMovie().getTitle()+ "\t" +
        String.valueOf(thisAmount) + "\n";
        totalAmount += thisAmount;
    }
    //add footer lines
    result += "Amount owed is " + totalAmount+"\n";
    result += "You earned " + frequentRenterPoints
            + " frequent renter points";
    return result;
}
```

(Satırlar ekrandan, 01:06:30, 01:20:00, 01:24:15 ve 01:31:30 karelerinden birleştirildi. `CHILDRENS` dalı aşağıda, `Movie`'ye taşınmış haliyle verilmiştir.)

9.3 Adım 1: Extract Method ile `amountFor`

`switch case` mantığını izole etmek için Extract Method kullanılır: bu kısım aynı sınıf içinde yeni bir metoda taşınır. Fowler'ın verdiği ad `amountFor`'dur; hoca daha güzel bir ad bulunabileceğini söyledi.

`switch`, filmin fiyat koduna bakıp `thisAmount` geçici değişkenine değer ekler. Döngünün sonunda toplam miktara `thisAmount` eklenir. O halde ayrı metot, o kira işleminden doğan borcu hesaplayıp döndürür. Parametre olarak yalnız film türü ya da film verilebilirdi; ama hesap gün sayısına da bağlı olduğu için hoca kira kaydının (`Rental each`) kendisini parametre olarak verdi. `thisAmount` değişkeni yalnız bu `switch` içinde yazılıyor, dışarıda yalnız okunuyor; bu yüzden değişken de yeni metoda taşındı ve metodun sonucu oldu (ekrandan, 01:20:00):

```
public double amountFor( Rental each ) {
    double thisAmount = 0;
    switch (each.getMovie().getPriceCode()) {
    case Movie.REGULAR:
        thisAmount += 2.0;
        if (each.getDaysRented() > 2)
            thisAmount += (each.getDaysRented() - 2) * 1.5;
        break;
    case Movie.NEW_RELEASE:
        thisAmount += each.getDaysRented() * 3.0;
        break;
    case Movie.CHILDRENS:
        ...
    }
    return thisAmount;
}
```

Sonra `statement()` içinde `thisAmount` kullanılan yerlerde `amountFor(each)` çağrıldı. Hoca geçici değişkenlerden hoşlanmadığı için `statement()` içinde yeni bir geçici değişken tutmadı; bu, `statement()` açısından bir Replace Temp with Query'dir.

Değişiklik yalnız sınıfın içinde kaldığı için test sınıfları değişmedi; yani yapılan iş "dışa verilen hizmetleri değiştirmeden iç yapıyı değiştirmek" ilkesine uydu. İstemeden bir hata oluşup oluşmadığını görmek için testler koşturuldu ve geçti.

Hoca Eclipse'in de refactoring yetenekleri olduğunu, bu işin menülerden de yapılabileceğini söyledi. Ders notları hazırlanırken Eclipse bu konuda çok yetenekli değildi, fazladan hatalar çıkarıyordu; bu yüzden işlemleri elle, tek tek yaptı. Her adımdan sonra hem sınıf şemasını hem sequence diagram'ı güncellediğini, sınıf şemasında bu adımda yalnız `amountFor` metodunun eklendiğini söyledi.

9.4 Adım 2: Yeniden adlandırma

Metot ve değişken adlarını değiştirmek küçük ve basit bir refactoring eylemidir, ama anlamayı kolaylaştırdığı için bir refactoring eylemi sayılır.

- `thisAmount` açıklayıcı bir ad değildir (“this” neyin “this”i?). Bu değer kira kaydından doğan borçtur ve metodun döndürdüğü sonuçtur; adı `result` yapıldı.
- Döngüde “her biri” anlamındaki `each` adı belki anlamlıdır, ama tek bir parametre alan metotta anlamlı değildir; parametre kira kaydını anlatan bir adla yeniden adlandırıldı.

Hoca bunun için Eclipse’in Refactor > Rename olanağını kullandı. Elle kopyala-yapıştır yapılsa bazı yerler atlanabilirdi; Eclipse adı her yerde kendisi değiştirir.

9.5 Adım 3: Move Method ile `Rental.getCharge()`

`amountFor` hâlâ uzundur ve `switch` hocayı rahatsız etmektedir; ama `switch`’i kaldırmak daha büyük bir değişiklik olduğu için sona bırakıldı. Hoca önce şuna dikkat çekti: bu metot parametre olarak bir kira kaydı alıyor ve sürekli o kira kaydını sorguluyor. Yani burada bir **Feature Envy** vardır. `Customer` sınıfının bu metodu `Rental` ile o kadar çok uğraşıyor ki aslında yeri `Rental` sınıfıdır. Müşteri kira kaydına “seni kiralamışım, bu işlem için borcum ne kadar?” diye sormalıdır.

Metot kesilip `Rental` sınıfına yapıştırıldı. Her kira kaydı kendisini bildiği için parametreye gerek kalmadı; `each.` önekleri silindi, metot kendi filminin fiyat koduna ve kendi gün sayısına bakar hale geldi. `switch` hâlâ duruyor, ama içindeki çağrılar artık nesnenin kendisiyle ilgili. `Customer` içinde çağrı `each.amountFor()` oldu (ekrandan, 01:24:15), sonra metot Rename ile `getCharge` diye adlandırıldı. Testler yine geçti.

Bu adımdan sonra `Customer` sınıfı 57-60 satırdan 40 satıra düştü.

Tartışma: `getCharge` için yeni bir test yazmalı mı? `Rental` yeni bir metot kazandı. İsteyen bunun için yeni bir test case yazabilir. Ama sınırlı insan kaynağıyla çalışılıyorsa mevcut test kapsamı yeterli görülebilir: mevcut testler zaten farklı türde filmler oluşturup farklı sürelerle kiralayarak bu hesabı sınıyor. Hoca, iç yapı değişti diye birim testlerini değiştirmenin beyaz kutu (white box) sınama yaklaşımında anlamlı olabileceğini, ama bu örnekte mevcut yaklaşıma devam ettiklerini söyledi.

9.6 Adım 4: Replace Temp with Query

`statement()` içindeki `thisAmount` yerel değişkeni gereksizdir: ona `getCharge` metodunun döndürdüğü değer atanmaktadır ve değişkenin geçerlilik süresince bu değer değişmemektedir (ekrandan, 01:30:00; slayt “Refactoring Örneği – 4. Müdahale”). Bu yüzden `thisAmount` yerine `each.getCharge()` kullanıldı (ekrandan, 01:31:30):

```
result += "\t" + each.getMovie().getTitle()+ "\t" +
String.valueOf(each.getCharge()) + "\n";
totalAmount += each.getCharge();
```

Slayta göre bedel hesaplama daha ileride gerçekleştiği için etkileşim (sequence) şeması da değişecektir.

`totalAmount` ise kümülatif toplam olduğu için bu aşamada kaldırılamaz. `frequentRenterPoints` de şimdilik kalır.

9.7 Adım 5: Sık kiralama puanı için `Rental.getBonus()`

Puan kazanma işi de kiralamayla ilgilidir. Madem `Rental`’a `getCharge` eklendi, bir de `getBonus` eklenebilir. Hoca “frequent renter points” yerine Türkçede yerleşmiş olan “bonus” kelimesini kullanmayı tercih etti; oyun terminolojisinde de kazanılan ek puanlara bonus denir.

Kural (ekrandan, 01:31:30): her kira işleminden 1 puan kazanılır; ek olarak film yeni çıkmış (`NEW_RELEASE`) ve 1 günden fazla kiralanmışsa 1 puan daha kazanılır. Hoca bunu Extract Method ile ayrı metoda çıkarırken daha çok kazanılan durumu önce yazdı, çünkü o durum azın şartını da içerir: koşul sağlanıyorsa 2, değilse 1 döndürülür. `each.getMovie()` önekleri silindi, metot kendi verisine bakar hale geldi.

`statement()` içinde iki satırlık puan artırma kodu ve onları açıklayan yorumlar yerine `frequentRenterPoints += each.getBonus();` yazıldı. Yorumlar da kaldırılabildi. Testler yine geçti.

Bu adımdan sonra `Rental` 40 satıra çıktı, `Customer` da ondan küçük hale geldi; daha önce en kalabalık sınıf `Customer` iken artık ikisi aynı düzeydedir.

9.8 Adım 6: `getTotalCharge()` ve `getTotalBonus()`

`statement()` içindeki `totalAmount` ve puan değişkenini doğrudan kaldırmak mümkün değildir; Replace Temp with Query yapmak gerekir. Bütün kira kayıtları `Customer`'da olduğu için bu sorgular da `Customer` sınıfına eklenir. Kümülatif toplamlar için iki metot yazıldı (ekrandan, 01:40:00):

```
public double getTotalCharge() {
    double result = 0;
    for( Rental aRental : _rentals )
        result += aRental.getCharge();
    return result;
}
public int getTotalBonus() {
    int result = 0;
    for( Rental aRental : _rentals )
        result += aRental.getBonus();
    return result;
}
```

Hoca döngüyü `while` ve `Enumeration` yerine for-each ile daha kısa yazdı. Toplam bedelde sonuç `double`, toplam bonusta `int`'tir. Geçici değişken `statement()`'tan kalktı, ama birikimci olduğu için yeni metotlarda ister istemez kaldı. `statement()`'ın altbilgi satırları `getTotalCharge()` ve `getTotalBonus()` çağrılarına dönüştü (ekrandan, 02:00:00). Toplam bedelin hesaplanması ve toplam bonusun hesaplanması kendi içinde anlamlı, ayrı işler olduğu için ayrı metot olarak kodlanmalarında bir zarar yoktur.

Başarım tartışması: Daha önce liste bir kez dolaşılırken artık üç kez dolaşılıyor: bir kez makbuz satırlarını yazmak için, birer kez de toplam bedel ve toplam bonus için. Kod toparlanıyor ve anlaşılır hale geliyor, ama başarım hissedilir derecede kötüleşiyor mu sorusu akla gelebilir. Hoca tüm testlerin tamamlanma süresine bakılırsa çok fark olmadığının görüleceğini söyledi.

Testler yine geçti. Bu adımdan sonra `statement()` 14 satıra indi. Artık odak `Customer`'dan `Rental` ve `Movie` sınıflarına kaydırılabilir.

9.9 Adım 7: `getCharge` hesabının `Movie`'ye taşınması

`Rental.getCharge()` içindeki `switch` hâlâ rahatsız edicidir. Bedeli belirleyen iki etken vardır: filmin ne kadar süre kiralandığı ve filmin türü. `Rental`'ın filmin türüyle bu kadar içli dışlı olması iyi değildir.

> **Hocanın vurgusu:** Her sınıf mümkün olduğu kadar kendi verisiyle çalışsın; ilişkide olduğu sınıfların verisiyle ilgili işlerin yapılmasını o sınıflara bıraksın. En doğrusu budur.

Metot tamamen taşınmaz, ikiye bölünür. `Movie` tek başına ne kadar borçlanılacağını bilemez; bilmesi gereken diğer bilgi kaç gün kiralandığıdır. O halde `Rental`, `Movie`'ye "bu kadar gün kiralandıysan borç ne olmalı?" diye sorar. `switch` bloğu `Movie`'ye taşındı, `getMovie()` önekleri silindi (film kendi fiyat koduna bakar), eksik bilgi olan gün sayısı `int days` parametresi oldu. `result` değişkenine Java'nın istediği için açıkça 0 ilk değeri verildi (ekrandan, 01:46:30):

```
public double getCharge(int days) {
    double result = 0;
    switch (getPriceCode()) {
    case Movie.REGULAR:
        result += 2.0;
        if (days > 2)
            result += (days - 2) * 1.5;
        break;
    case Movie.NEW_RELEASE:
```

```
            result += days * 3.0;
            break;
        case Movie.CHILDRENS:
            result += 1.5;
            if (days > 3)
                result += (days - 3) * 1.5;
            break;
        }
        return result;
    }
```

(Metot imzası ve `switch` satırı karede görünmüyor; altyazıdaki anlatıma göre yazıldı. Karede görünen kısım 27-39. satırlardır.)

`Rental.getCharge()` artık yalnız bir köprü metottur ve geçici değişkene gerek kalmamıştır (ekrandan, 01:50:00):

```
public double getCharge( ) {
    return _movie.getCharge(_daysRented);
}
```

Testler geçti. Boyutlara bakıldığında `Rental` ile `Movie` adeta yer değiştirdi. Hoca olayın yalnız büyüklük olmadığını, her türlü verinin ihtiyaç duyulduğu ve doğru yerde kullanılmasına doğru ilerlendiğini söyledi.

9.10 Adım 8: `getBonus` hesabının `Movie`'ye taşınması

Aynı mantıkla bonus da bölündü. `getCharge` kadar göze batmasa da `Rental.getBonus()` da `Movie`'nin bir üye alanına doğrudan erişiyordu. Metot `Movie`'ye kopyalandı; gün sayısı `days` parametresi oldu, film kendi fiyat koduna bakar: fiyat kodu yeni çıkan film ve `days` 1'den büyükse 2, değilse 1 döndürülür. `Rental` tarafı yine köprü metot oldu (ekrandan, 01:50:00):

```
public int getBonus( ) {
    return _movie.getBonus(_daysRented);
}
```

Testler geçti.

9.11 Adım 9: `Rental.getMovieTitle()` ve sequence diagram'ın yararı

Hoca sequence diagram çizmenin bir avantajını gösterdi: sınıf şemasında fark edilmeyecek bir şey burada görülür. Etkileşim incelendiğinde `Customer` hep `Rental`'a sorar; `Rental` işi kendisi halleder ya da bir kısmı için `Movie`'den yardım ister. Ama bir istisna vardır: makbuz basılırken filmin adını öğrenmek için `Customer` doğrudan `Movie`'ye (`getTitle`) sorar.

Daha iyi bir katmanlama için filmin adı da `Rental`'a sorulmalıdır. `Rental`'a `getMovieTitle()` metodu eklendi (içinde `_movie.getTitle()` döner) ve `statement()` içinde `each.getMovie().getTitle()` yerine `each.getMovieTitle()` yazıldı (ekrandan, 02:00:00). Testler geçti. Sonuçta `Customer`'dan `Movie`'ye doğrudan giden bir ok kalmadı; yalnız dolaylı bağlaşımlar var. Bu durum sınıf şemasında görünmez, ama önceki ve sonraki sequence diagram'lar karşılaştırıldığında görülür.

9.12 Adım 10: `switch` yerine çok biçimlilik (yapılmadı, tartışıldı)

Geriye `switch case`'in nasıl kaldırılacağı kaldı. Değişik film türleri olduğu en başta bilindiğinde yapılması gereken tasarım şudur (ekrandan, 01:53:30; slayt "Refactoring Örneği – 10. Müdahale"): `getCharge` metodundaki `switch/case` ve `getBonus` metodundaki `if` deyimleri yerine kalıtım ve soyut metotlarla çok biçimlilik kullanılır.

```
Customer                        Rental                          <<abstract>> Movie
+ statement()                   - _daysRented : int             + getCharge(daysRented : int)
+ getTotalCharge() : double     + getCharge() : double          + getBonus(daysRented : int) :
+ getTotalBonus() : int         + getBonus() : int
                                + getMovieTitle() : String
```

```
Customer 1 ---- * Rental   (Customer'dan Rental'a yönlü)
Rental   1 ---> 1 Movie
Movie <|-- MovieNewRelease, MovieRegular, MovieChildrens
```

`Movie` soyuttur, çünkü genel bir "film" türü yoktur; her film bu türlerden biridir. İstenirse `Movie` soyut yapılmaz, normal sınıf olur ve adı `RegularMovie` olur; yeni çıkan ve çocuk filmleri ondan kalıtımla türetilir. Her film kendi türünü bildiği için `switch`'in her `case`'i uygun alt sınıfa taşınır ve `switch` tamamen ortadan kalkar.

Hoca en başta böyle yapılması gerektiğini, ama önceki tasarımla devam edilmişse ve geriye doğru uyumluluk korunmak isteniyorsa bu düzenlemenin yapılamayacağını söyledi: yepyeni üç sınıf ortaya çıkar ve dış metotlar açısından değişmese bile birim testleri ister istemez değişir, çünkü testlerde film nesneleri yeni türlerden oluşturulmalıdır. Bu yüzden örnek bu noktada bırakıldı. `switch`'in dezavantajı da bir ölçüde en aza indirilmiştir: `case`'lerdeki her şey artık nesnenin kendisiyle ilgilidir; fiyat kodu kendisindedir, gün sayısı da parametre olarak gelir.

Hoca şunu da ekledi: ürün piyasaya yeni çıktıysa ve geriye uyumluluk çok dert değilse, test case'ler de değiştirilebiliyorsa daha iyi tasarıma ilerlemek için bu adım yapılabilir; değilse yapılmayabilir. Bir ara seçenek olarak **Replace Type Code with Strategy** de yapılabilir; bu durumda da test case'ler değişir ama belki daha az ölçekte. Hoca bunu isteğe bıraktı.

> **Ek bilgi (derste anlatılmadı):** Fowler'ın kitabının 1. bölümünde bu son aşama büyük olasılıkla State kalıbıyla yapılır: `Movie` bir `Price` nesnesine bağlanır ve `RegularPrice`, `NewReleasePrice`, `ChildrensPrice` alt sınıfları `getCharge` hesabını gerçekler. Böylece bir filmin fiyat türü çalışma anında değişebilir (yeni çıkan bir film zamanla normal filme dönüşür). Bu, 6. bölümdeki Replace Type Code with State/Strategy eyleminin uygulamasıdır.

9.13 Ek adım: `addFooter` ile altbilginin çıkarılması

Yorum satırlarından mümkün olduğunca kurtulmak hedeflendiği için `statement()` içindeki "add footer lines" yorumuyla ayrılmış altbilgi yazma işi Extract Method ile ayrı bir metoda çıkarıldı. Hoca önce `printFooter` adını düşündü, metot `String` döndürdüğü için `addFooter` adını uygun buldu (ekrandan, 02:00:00). Yorum satırı silindi ve `statement()` içinde altbilgi `result += addFooter();` gibi tek bir komutla eklendi. Hoca yeni metodun birkaç satıra yayılmasının yalnız biçim tercihi olduğunu, aslında tek bir komut olduğunu söyledi. Testler geçti ("all green").

10. Kod Kusurlarından Anti-Pattern'lere

Hoca derste neler öğrenildiğini hatırlattı: önce tasarım kalıpları (design patterns), sonra nesneye yönelimin doğru kullanılmaması vakaları olan küçük ölçekli kod kusurları (code smells). Bu kusurların düzeltilmesinde en fazla Extract Method, Extract Class gibi eylemlerle bir yeni sınıf çıkarıldı; iki, üç ya da daha fazla sınıflık yapılarla uğraşılmadı.

Soru şudur: kod kusurları daha büyük ölçekte olabilir mi? Bir tasarım kalıbı gibi çok sayıda sınıf içeren bir yapıya genişletilebilir mi? Bu konu **anti-pattern** (karşıt kalıp) konusudur. Konuyla ilgili güzel bir kitap ve web siteleri vardır. Karşıt kalıplar üç alt gruba ayrılır:

- Yazılım geliştirme karşıt kalıpları
- Yazılım mimarisi karşıt kalıpları (bu iki grup tasarım kalıplarının tam tersidir)
- Yazılım proje yönetimi karşıt kalıpları: yazılım proje yönetiminde önerilen uygulamaların tersini yapmak

Hoca isteyenlerin bu konuyu araştırabileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Bu üçlü sınıflandırma büyük olasılıkla Brown, Malveau, McCormick ve Mowbray'in 1998 tarihli "AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis" kitabına dayanır. Örneğin her şeyi bilen ve yapan dev bir sınıf olan "The Blob" (God Class) bir yazılım geliştirme karşıt kalıbıdır ve Large Class kod kusurunun büyük ölçekli karşılığı gibi düşünülebilir.

Örnek Sorular

Dersin sonunda birinci ara sınavın akademik soruları tartışıldı. Soru metinleri ekrandan alınmıştır.

Soru 1 (ekrandan, 02:02:10): Bir işletim sisteminin komut satırı ortamında (shell) aşağıdaki gibi bir çalışma biçimi kurulacaktır:

```
Unix benzeri: ps -ef | grep $process | sort | sendmail
DOS benzeri:  dir /w > duzenle.exe > blat.exe /verbose
```

Böyle bir çalışma düzenini kurabilmek için hangi tasarım kalıbını kullanırsınız? Nedenlerini de söyleyerek açıklayınız. Ardından tasarımınızın UML sınıf şemasını çiziniz. Not: Örneklerdeki komutların ne yaptığının önemi yoktur; bir komutun çıktısının diğerine yönlendirildiğini bilmek yeterlidir.

> **Çözüm (derste sözlü):** Hoca bir komutun çıktısının bir sonrakinin girdisi olduğu bu yapı için Decorator kalıbını önerdi. İşletim sistemi sıfırdan tasarlanıyorsa her komut, yapılmak istenen iş zincirine yeni bir yetenek ekler; bu Decorator'ün yapısına uyar. Hoca örnek olarak bir Unix makinede sorun yaşandığında teknik desteğin, o anda çalışan süreçlerin listesini süzüp sıralayıp e-postayla göndermek için böyle bir satır yazdırabileceğini anlattı. Başka bir kalıp seçip onun bu senaryoda nasıl gerçeklenebileceğini doğru açıklayan yanıtları da değerlendirebileceğini söyledi.

Soru 2 (ekrandan, 02:05:40): Bir Java arayüzü, çeşitli metot imzalarını bu metotların gerçeklemeleri (gövdeleri, kodu) olmadan ve çoklu kalıtımın sakıncalarını ortaya çıkarmadan bir araya getiren bir yapıdır. Örneğin:

```
public interface TelefonSahibi {
    public String getTelefon( );
    public void setTelefon( String telefon );
}
```

Buna göre Java arayüzleri Template Method tasarım kalıbındaki üst sınıf yerine kullanılabilir mi? Neden(ler)i ile birlikte açıklayınız.

> **Çözüm (derste sözlü):** Kullanılamaz. Template Method'daki üst sınıfta bazı metotlar soyut, bazıları somuttur; bütün metotlar soyut değildir. Özellikle template metodun gövdesinin üst sınıfta kodlanması gerekir. Metot gövdesi alamayan bir arayüz bu yüzden burada kullanılamaz.
>
> **Ek bilgi (derste anlatılmadı):** Java 8'den beri arayüzlerde `default` metotlarla gövde yazılabilir. Yine de arayüz metotları `final` yapılamadığı için alt sınıfların template metodu değiştirmesi engellenemez; arayüzler üye alan (durum) da tutamaz. Bu yüzden Template Method için soyut üst sınıf hâlâ doğal seçimdir.

Soru 3 (ekrandan, 02:07:30): Aşağıdaki UML şeması ile gösterilen sistemde hangi tasarım kalıbı kullanılmıştır? Açıklayınız.

```
<<interface>> Test
  + run(TestResult : int) : void
       ^                  ^
       :  (gerçekler)     :  (gerçekler)
TestCase                  TestSuite  ----------------> * Test
  + run(TestResult:int)     + run(TestResult : int) : void
                            + addTest(Test : int) : void
```

> **Çözüm (derste sözlü):** Composite kalıbı. Bir test kümesi (`TestSuite`) test ekleyebilir ve kendi gerçeklediği `Test` arayüzü türünden parçalarla bir parça-bütün ilişkisi kurar; tek test (`TestCase`) de aynı arayüzü gerçekler. Parça-bütün ilişkisi ve parçaların bütünle aynı arayüzü paylaşması Composite'in yapısıdır.

Soru 4 (ekrandan, 02:07:30): Aşağıdaki UML şeması ile gösterilen sistemde hangi tasarım kalıbı kullanılmıştır? Açıklayınız.

```
Client ---------------------> Collection
   |                             + createTraversalObject() : TraversalAbstraction
```

```
   v                                  ^                 ^
TraversalAbstraction           ListCollection      MapCollection
  + first()                      + createTraversalObject()
  + next()                         (not: return new ListTraverser( this );)
  + isDone()
     ^           ^
MapTraverser   ListTraverser
```

> **Çözüm (derste sözlü):** Iterator kalıbı. Farklı koleksiyon türleri kalıtımla türetilmiştir; dolaşmayı soyutlayan başka bir sınıf ve onun alt sınıfları vardır. Böylece veri yapısının saklanma biçimi ile dolaşılma biçimi ayrı ayrı özelleştirilebilir; yani ayrı ayrı özelleşmiş iki kalıtım hiyerarşisi bir araya getirilmiştir. Hoca Bridge yanıtının çok doğru olmayacağını söyledi, çünkü Bridge'de iki alt hiyerarşinin kökleri arasında doğrudan bir ilişki olur; asıl düşündüğü yanıt Iterator'dır.

Dersin Toparlaması

Bu derste refactoring konusunun son kod kusurları işlendi. Divergent Change ve Shotgun Surgery, bir değişikliğin bir sınıfın birçok metoduna ya da birçok sınıfa yayılmasıdır ve Extract Class, Move Method, Move Field ile düzeltilir. Primitive Obsession, ilkel tiplerin aşırı kullanımıdır; veri değeri bir nesneye, tip kodu bir sınıfa (ya da `enum`'a), alt sınıflara ya da State/Strategy kalıbına, dizi de bir nesne dizisine çevrilerek düzeltilir. Feature Envy, bir metodun başka sınıfın üyeleriyle kendi sınıfınınkinden fazla uğraşmasıdır ve Move Method ya da Extract Method ile düzeltilir; Strategy ve Visitor bu kusur sayılmaz.

Dersin ikinci yarısında Fowler'ın video kiralama örneğinde bu eylemler sırayla uygulandı. `Customer.statement()` içindeki uzun `switch` önce `amountFor` olarak çıkarıldı, Feature Envy fark edilince `Rental.getCharge()` oldu, sonra hesabın film türüne bağlı kısmı `Movie.getCharge(days)` olarak ayrıldı. Bonus için de aynı yol izlendi, toplamlar sorgu metotlarına dönüştü, `Customer`'ın `Movie`'ye doğrudan bağımlılığı `getMovieTitle()` köprüsüyle kaldırıldı. Her adımda test kümesi değişmeden geçti. Son adım olan `switch` yerine film alt sınıfları kullanmak birim testlerini değiştireceği için yalnız tartışıldı. Hoca, kod kusurlarının daha büyük ölçekli karşılığı olan anti-pattern'leri araştırma konusu olarak önerdi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.